



Timeline



Título del tfg

Diseño, desarrollo y despliegue de aplicación web full-stack para el análisis de estadísticas de videojuegos mediante el uso de API externa.

Riot Api



DEVELOPER

[APIs](#)

[DOCS](#)

[POLICIES](#)

[FAQS](#)

[CHANNELS](#)

[SUPPORT](#)

ACCOUNT-V1

Riftbound RSO

CHAMPION-MASTERY-V4

League of Legends

CHAMPION-V3

League of Legends

CLASH-V1

League of Legends

LEAGUE-EXP-V4

League of Legends

LEAGUE-V4

League of Legends

LOL-CHALLENGES-V1

League of Legends

LOL-RSO-MATCH-V1

League of Legends RSO

LOL-STATUS-V4

League of Legends

LOR-DECK-V1

Legends of Runeterra RSO

LOR-INVENTORY-V1

Legends of Runeterra RSO

There are three routing values for account-v1; americas, asia, and europe. You can query for any account in any region. We recommend using the nearest cluster.

ACCOUNT-V1

GET /riot/account/v1/accounts/by-puuid/{puuid}

[Jump to Inputs](#)

RESPONSE CLASSES

Return value: AccountDto

AccountDto

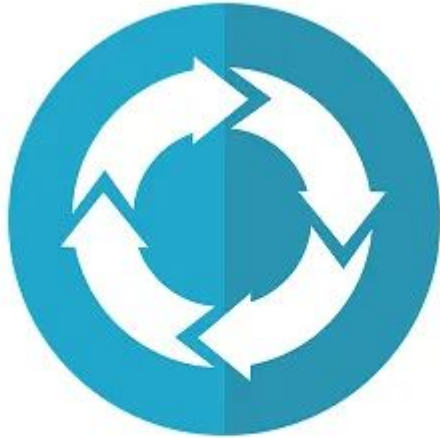
NAME	DATA TYPE	DESCRIPTION
puuid	string	Encrypted PUUID. Exact length of 78 characters.
gameName	string	This field may be excluded from the response if the account doesn't have a gameName.
tagLine	string	This field may be excluded from the response if the account doesn't have a tagLine.

RESPONSE ERRORS

HTTP STATUS CODE	REASON
400	Bad request
401	Unauthorized

[Pincha aquí](#)

Ciclo de vida



Iterativo / Incremental

¿Por qué iterativo incremental?

- **Desconocimiento de las tecnologías:** Se desconocen muchas tecnologías + funcionamiento de apis Rest.
- **Desconocimiento del alcance:** Al poder tener funcionalidades prácticamente ilimitadas, se desconoce el alcance exacto del producto.

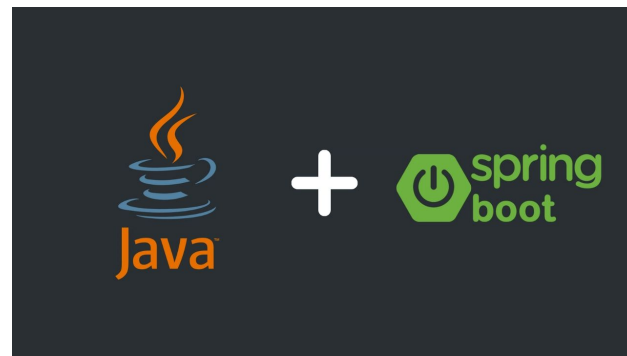
Stack Inicial

Investigación Previa

Backend - Spring Boot

Motivos:

- **Framework de java:** Spring Boot es el framework más utilizado que existe de java. A su vez, java es el lenguaje principal que se estudia en la carrera.
- **Framework popular:** Es el framework estándar empresarial de java. Además, es uno de los frameworks más populares en la actualidad, con un 15% de uso entre todos los desarrolladores.
- **Framework conocido:** Es el framework de backend que mejor conozco para realizar el tfg de manera adecuada.



- Model
- Repository
- Service
- Jobs
- Controller
- Riot (Llamadas api Riot)

[Fuente aquí](#)

Testing - JUnit

Motivos:

- **Estándar de java:** JUnit es el framework de testing estándar de java, por lo que como el backend es de java, los tests también.



[Fuente aquí](#)

BD Principal - PostgreSQL

Motivos:

- **BD más popular:** PostgreSQL es la base de datos más popular en las empresas hoy en día y en lo seguirá siendo en el futuro.
- **Open Source:** Además, es de código abierto, no requiere licencia alguna y la comunidad la mejora constantemente.
- **Optimización y eficiencia:** Gestiona consultas complejas e índices avanzados para acceder a los datos de forma más rápida. Además, es fácilmente escalable.



[Fuente aquí](#)

[Fuente 2 aquí](#) (Aquí compara con otras db, no se si incluir esto)

BD Secundaria - Redis

Motivos:

- **Necesidad de BD en memoria:** Para evitar consultas repetitivas al backend se necesita implementar una BD caché en memoria.
- **BD más popular:** Es la 5º BD más utilizada por desarrolladores profesionales. Además, su uso crece cada año.
- **Flexibilidad:** Redis no almacena solo pares clave-valor, también soporta estructuras complejas como Strings, listas, hashes... Esto la hace más versátil que el resto de opciones (memcached).



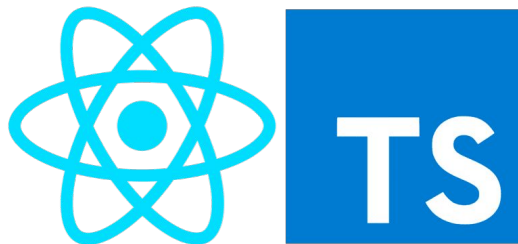
[Fuente 1 aquí](#)

[Fuente 2 aquí](#)

Frontend - React + TS

Motivos:

- **React:** React es el framework más popular (44.7%) y más deseado entre todos los frameworks de frontend.
- **Ventajas React:** Para cuando me ponga con ello. Solo se que es el más popular por su curva de aprendizaje.
- **Ventajas Typescript:** Para cuando me ponga con ello. Se que es javascript (la única otra opción) pero con esteroides. Se usa por su tipado estático y se compila a js.



[Fuente aquí](#)

Framework CSS - Tailwind

Motivos:

- **Desarrollo más rápido:** Tailwind ofrece clases (plantillas) listas para usar directamente en html permitiendo diseñar interfaces de forma sencilla.
- **Limpieza y mantenimiento:** El uso de tailwind permite eliminar largas hojas CSS mejorando la organización del proyecto y evitando el CSS repetitivo.
- **Popularidad y Flexibilidad:** Es una de las herramientas más populares en el desarrollo frontend junto con Bootstrap. Sin embargo, tailwind no impone un diseño fijo, sino que permite crear interfaces personalizadas.



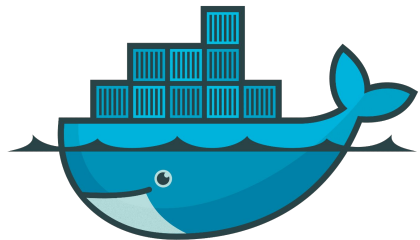
[Fuente 1 aquí](#)

[Fuente 2 aquí](#)

Despliegue - Docker

Motivos:

- **Portabilidad:** Docker garantiza que la aplicación siempre se ejecute de la misma forma a partir de contenedores.
- **Facilidad al desplegar:** Para desplegar una aplicación simplemente hay que ejecutar los contenedores sin necesidad de instalar nada.
- **Aislamiento:** Permite separar cada componente en su propio contenedor. En nuestro caso: Backend, frontend, postgresQL y Redis.



docker

- Backend
- Frontend
- PostgreSQL
- Redis

[Fuente aquí](#)

CI/CD - GitHub Actions

Motivos:

- **Automatización:** Automatiza build + tests en cada push reduciendo fallos en despliegues.

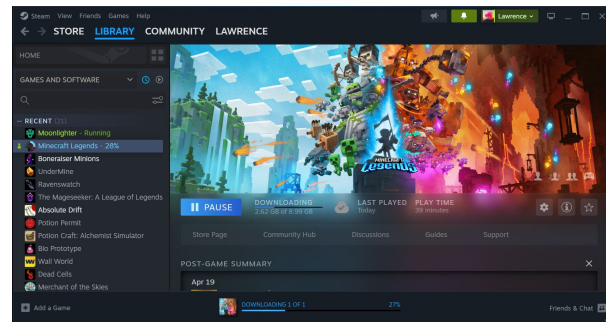
Más, cuando llegue a aprenderlo



[Fuente aquí](#)

App de escritorio (Opcional) - No conocido

De esto de momento me olvido, se rellenará si finalmente se hace.



Iteración 1

Diagrama CDU

Decisiones:

Para la primera iteración, sólo se añadirán las funcionalidades básicas (consulta de perfiles y builds recientes). De esta forma se asegura el correcto funcionamiento del ciclo de vida completo sin entrar en detalles técnicos más complejos.

Las operaciones que implican analizar miles de partidas (Build recomendada por winrate) se dejan para las iteraciones siguientes.

*Build: Conjunto de ítems que compra un jugador.

Creo que habrá que explicar conceptos previos en el pdf para que se entienda el contexto.

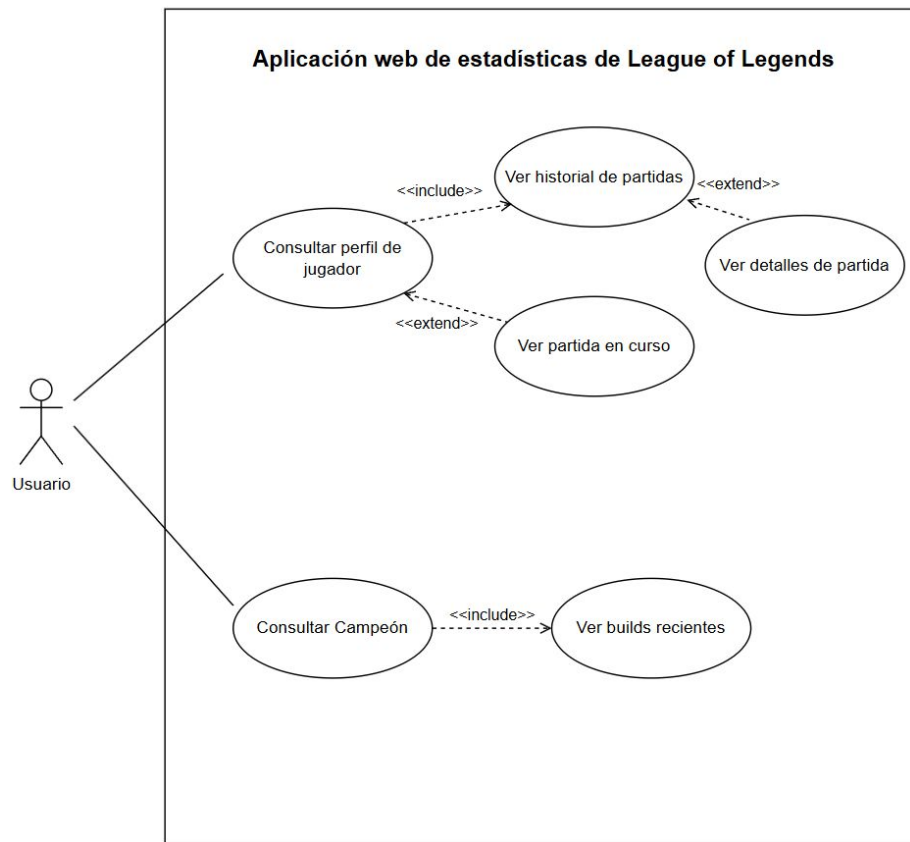


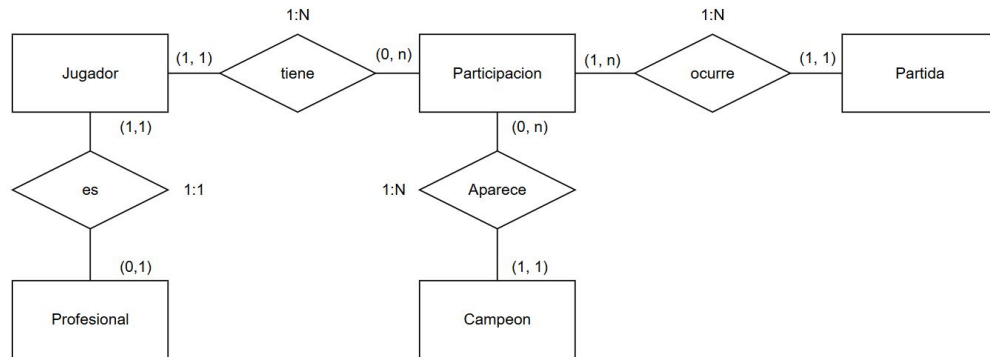
Diagrama E-R

Decisiones:

Cuando un jugador juega una partida, utiliza un campeón (personaje). Para evitar una relación triple con atributos (relación asociativa) que, en un diagrama E-R se ve muy abstracto, decidí crear una entidad intermedia "Participación" que une todas las demás.

Esta entidad habría aparecido de todas formas en el paso al relacional, pero como no se va a realizar lo diseñé así desde el principio para evitar problemas.

Diagrama E-R



Diseño de entidades en Spring Boot

```
4
5 import java.time.Instant;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 @Entity & fmonteroi
10 public class Player {
11
12     @Id
13     private String puuid;
14
15     private String riotGameName; 2 usages
16     private String riotTagLine; 2 usages
17
18     @Enumerated(EnumType.STRING) 2 usages
19     private PlatformRegion platformRegion;
20
21     private Integer profileIconId; 2 usages
22     private Integer summonerLevel; 2 usages
23     private Instant updatedAt; 2 usages
24
25     // Relations
26     @OneToOne(mappedBy = "player", cascade = CascadeType.ALL, orphanRemoval = true) 2 usages
27     private Professional professional;
28
29     @OneToMany(mappedBy = "player", cascade = CascadeType.ALL, orphanRemoval = true) 2 usages
30     private List<Participation> participations = new ArrayList<>();
31
32     public Player() { & fmonteroi
33     }
34
35     > public String getPuuid() { return puuid; }
36
37     > public void setPuuid(String puuid) { this.puuid = puuid; }
38
39     > public String getRiotGameName() { return riotGameName; }
40
41     > public void setRiotGameName(String riotGameName) { this.riotGameName = riotGameName; }
42
43     > public String getRiotTagLine() { return riotTagLine; }
```

Decisiones:

Se diseñaron todas las entidades y relaciones del E-R.

Se tomó la decisión de realizar todo el código y documentación interna en inglés, ya que es el lenguaje usado globalmente.

```
Player.java x Professional.java x RoutingRegion.java x
1 package es.unex.cume.tfg.backend.model;
2
3 import jakarta.persistence.*;
4
5 import java.time.Instant;
6
7 @Entity & fmonteroi
8 public class Professional {
9     @Id
10     private String puuid;
11
12     private String teamName; 2 usages
13     private String league; 2 usages
14     private Instant lastBuildUpdateAt; 2 usages
15
16     // Relations
17     @OneToOne(fetch = FetchType.LAZY) 2 usages
18     @MapsId
19     private Player player;
20
21     public Professional() { & fmonteroi
22     }
23
24     public String getPuuid() { & fmonteroi
25         return puuid;
26     }
27
28     public void setPuuid(String puuid) { & fmonteroi
29         this.puuid = puuid;
30     }
31
32     public String getTeamName() { no usages & fmonteroi
33         return teamName;
34     }
35 }
```

Aquí paro ya -> ver correo